

A Comparison of Symbolic and Neuro-Symbolic Strategies for LTLf Runtime Monitoring

Author name

Date

Abstract

TODO

1 Introduction

Linear Temporal Logic (LTL) [10] is a logic formalism for expressing temporal properties of reactive systems. It extends traditional propositional logic with modal operators, allowing the specification of rules that must hold *through time*, and it is interpreted over traces of discrete events. Variants of LTL interpreted over finite traces have been proposed for modeling finite, but length-unbounded, process executions (see Sec. 2.1), making it suitable for finite-horizon problems that often arise in the context of AI and BPM. Temporal logics have provided one of the most popular tool in Formal Methods for verification of hardware or software system behaviors. In particular, model checking is the problem of verifying whether a model of a system meets a given specification. Despite the importance and the progress made in model checking, it is not always possible to check the model against the specification. This is due to the inherent complexity of the system or the fact that the model is only partially correct or not known. In such cases, one can resort to simpler verification approaches, such as runtime monitoring, where the system’s execution is continuously observed on-the-fly and any permanent violation or fulfillment of the specified property is immediately detected.

Standard symbolic techniques for both model checking and runtime monitoring have achieved great success, but they also suffer for some inherent limitation. Among the others there is the difficulty in dealing with the increasing growth of big data and the lack of flexibility in dealing with highly unpredictable environments, say e.g. environments with noisy sensors or probabilistic uncertainties, and adapting the specification when soft violations could happen. Conversely, pure deep learning architectures, such as Recurrent Neural Networks (RNNs) and Transformers, excel at modeling high-dimensional, noisy data distributions, but they likely generate outputs that violate critical safety constraints, as they lack a mechanism to adhere to formal logics. This naturally leads to the use of hybrid approaches, which can possibly overcome some of the mentioned limitations.

In this paper, we will compare different techniques for runtime monitoring, from purely symbolic methods to the state-of-the-art neuro-symbolic architectures. We start revisiting the neural-symbolic approach to runtime monitoring, originally proposed by [8, 9] through the RuleRunner system [7]. The authors claimed that the resulting neural monitors could exploit GPU parallelism for efficient verification, outperforming purely symbolic mechanisms. Since then, some significant developments have reshaped the landscape. First, LTL_f [2] has become the standard formalism for

temporal reasoning over finite traces in AI, bringing with it mature tools for compiling formulas into DFAs. Then, differentiable automata representations such as DeepDFA [11] have emerged, enabling the integration of temporal logic knowledge into neural architectures through gradient-based learning. **Possibly include other NeSy approaches.** We present a systematic comparison of three paradigms for LTL_f runtime monitoring: (i) purely symbolic DFA-based monitoring, (ii) rule-based neural-symbolic monitoring in the RuleRunner tradition, and (iii) automata-based neural-symbolic monitoring using DeepDFA.

TODO: Mention briefly the experimental results obtained.

2 Background

2.1 An Historical Overview of Temporal Logics over Finite Traces

Studying LTL over finite traces gained significant attention [4, 5]. The original RuleRunner system [7] refers to FLTL as a finite-trace variant of LTL. As already mentioned, in the last decade LTL_f has established as the standard language for modeling finite traces of executions. Notably, FLTL and LTL_f share essentially the same syntax and semantics, offering two next operators, with a strong next operator X which is false at the last position if no successor exists, and a weak next operator W which is true at the last position (see Sec. 2.2 for details). FLTL makes explicit use of an *END* symbol for signaling the last element of the trace, but this can be easily expressed with the weak next operator by $END \equiv W\text{false}$. In this paper, we use LTL_f as the specification language for all three monitoring paradigms we compare.

TODO: Mention other finite-trace variants.

2.2 Syntax and Semantics

Given a finite set Σ of atomic propositions, an LTL_f formula φ has the form:

$$\varphi ::= \top \mid \perp \mid a \mid \neg\varphi \mid \varphi_1 \wedge \varphi_2 \mid X\varphi \mid \varphi_1 U \varphi_2,$$

where $a \in \Sigma$. We use $\top$ and $\perp$ to denote *True* and *False*, respectively. X (Strong Next) and U (Until) are temporal operators. Other temporal operators can be derived as follows: W (Weak Next) is defined as $W\varphi \equiv \neg X\neg\varphi$, R (Release) is defined as $\varphi_1 R \varphi_2 \equiv \neg(\neg\varphi_1 U \neg\varphi_2)$, G (globally) is defined as $G\varphi \equiv \perp R \varphi$, and F (eventually) is defined as $F\varphi \equiv \top U \varphi$.

A trace $\sigma = (\sigma_1, \sigma_2, \dots, \sigma_T)$ is a sequence of propositional assignments to the propositions in Σ , where $\sigma_t \in 2^\Sigma$ is the set of all and only propositions that are true at instant t . Additionally, $|\sigma| = T$ denotes the length of the trace. Note that, if the propositional symbols in Σ are all *mutually exclusive*, i.e., the domain produces exactly one symbol true at each step, then we have $\sigma_t \in \Sigma$. Let $last = |\sigma| - 1$. We inductively define when an LTL_f formula φ is true at an instant i (for $0 \leq i \leq last$), written $\sigma, i \models \varphi$, as follows:

- $\sigma, i \models a$ iff $a \in \sigma_i$, for $a \in \Sigma$;
- $\sigma, i \models \neg\varphi$ iff $\sigma, i \not\models \varphi$;
- $\sigma, i \models \varphi_1 \wedge \varphi_2$ iff $\sigma, i \models \varphi_1$ and $\sigma, i \models \varphi_2$;
- $\sigma, i \models X\varphi$ iff $i < last$ and $\sigma, i + 1 \models \varphi$;

- $\sigma, i \models \varphi_1 \mathcal{U} \varphi_2$ iff for some j such that $i \leq j \leq \text{last}$, we have $\sigma, j \models \varphi_2$, and for all k such that $i \leq k < j$, we have $\sigma, k \models \varphi_1$.

We say that σ satisfies ϕ , written $\sigma \models \varphi$, if $\sigma, 0 \models \varphi$. A formula φ is satisfiable if it is true wrt some trace σ , and is valid, if it is true wrt every trace σ .

Any LTL_f formula φ can be translated into a Deterministic Finite Automaton (DFA) [2] $A_\varphi = (2^\Sigma, Q, q_0, \delta, F)$, where 2^Σ is the automaton alphabet, Q is the finite set of states, $q_0 \in Q$ is the initial state, $\delta : Q \times 2^\Sigma \rightarrow Q$ is the transition function, and $F \subseteq Q$ is the set of final states. Additionally, we recursively define the extended transition function over traces $\delta^* : Q \times (2^\Sigma)^* \rightarrow Q$ as:

$$\begin{aligned} \delta^*(q, \epsilon) &= q \\ \delta^*(q, \sigma + \mathbf{x}) &= \delta^*(\delta(q, \sigma), \mathbf{x}), \end{aligned} \tag{1}$$

where $\sigma \in 2^\Sigma$ is a symbol and $\mathbf{x} \in (2^\Sigma)^*$ is a trace. The automaton accepts the trace σ iff $\delta^*(q_0, \sigma) \in F$, and in that case we say that σ belongs to the language of the automaton, denoted as $\mathcal{L}(A_\varphi)$. We have that φ and A_φ are equivalent because, for any trace $\sigma \in (2^\Sigma)^*$:

$$\sigma \in \mathcal{L}(A_\varphi) \iff \sigma \models \varphi. \tag{2}$$

3 Symbolic and Neuro-Symbolic Monitoring Strategies

3.1 Symbolic Monitoring

Architecture. The purely symbolic strategy is the classical automata-theoretic approach to runtime monitoring, and it serves as the exact baseline against which the two neuro-symbolic paradigms are measured. Given a specification φ , we compile it once into the minimal DFA $A_\varphi = (2^\Sigma, Q, q_0, \delta, F)$ introduced in Section 2.2, using the standard LTL_f -to-automaton route [2, 12] as implemented by `ltlf2dfa` [3]. Monitoring is then a deterministic walk over this automaton: the monitor keeps a single current state q , initialized to q_0 , and on observing the cell $\sigma_t \in 2^\Sigma$ it advances $q \leftarrow \delta(q, \sigma_t)$. Because the automaton is fixed and deterministic, each step costs a single transition lookup, independent of the trace seen so far and of the size of φ ; after processing a prefix $\sigma_1 \cdots \sigma_t$ the current state is exactly $\delta^*(q_0, \sigma_1 \cdots \sigma_t)$. This makes symbolic monitoring the fastest paradigm on a single trace, at the cost of being entirely static: the automaton encodes the specification exactly and admits no learning or adaptation.

Verdicts and early termination. Online monitoring requires the three-valued runtime semantics of [1]: at each cell the monitor emits SATISFY, VIOLATE, or UNDECIDED, the last meaning that the observed prefix can still be extended into both an accepting and a non-accepting continuation. A definite verdict can be issued as soon as the outcome is settled regardless of the future, which corresponds to two structural properties of the current state, both precomputed once by backward reachability over A_φ . A state is a *trap* if no accepting state is reachable from it: reaching a trap means φ is permanently violated, so the monitor halts with VIOLATE. Dually, a state is an *accepting sink* if every state reachable from it (including itself) is accepting: the monitor halts with SATISFY. With trap and sink sets materialized at compile time, the per-step verdict reduces to two set-membership tests on top of the transition lookup. As long as the current state is neither, the verdict is UNDECIDED and monitoring continues.

Some specifications never reach a trap or a sink, so STEP returns UNDECIDED for the entire trace; the canonical example is the response pattern $G(a \rightarrow Fb)$, which can always be satisfied by a later b and always be rescued from violation, hence has neither an accepting sink nor a trap. For these the verdict is only binary at the end of the trace, and the monitor reports a final verdict by testing acceptance of the state reached, i.e. SATISFY iff $\delta^*(q_0, \sigma) \in F$ and VIOLATE otherwise. Crucially, this monitor is exact on every LTL_f formula: its verdicts coincide with the semantics of Section 2.2 by construction, including the nested-temporal specifications on which RuleRunner diverges (Section 3.2).

3.2 Rule-based NeSy Monitoring: RuleRunner

Architecture. A RuleRunner system is a tuple $\langle S, R_E, R_R \rangle$, where S is the state of the system, R_E is the set of *evaluation rules*, and R_R is the set of *reactivation rules*. In details:

- The state S contains observations, active rules, and truth evaluations. $R[\phi]$ denotes an active rule, while $[\phi]V$ denotes a 3-values truth evaluation, with $V \in \{T, F, ?\}$.
- Evaluation rules are used to compute the truth value of a formula ϕ under verification, given the truth values of its direct subformulae ψ_i . They have the form $R[\phi], [\psi_1]V, \dots, [\psi_n]V \rightarrow [\phi]V$. For instance, $R[a \vee b], [a]T \rightarrow [a \vee b]T$, because the disjunction is evaluated to *true* if one of the two disjuncts is true.
- Reactivation rules are used when a formula ϕ is evaluated to undecided, i.e. $[\phi]?$, in which case that formula and its subformulae are scheduled to be monitored again in the next cell of the trace. They have the form $[\phi]? \rightarrow R[\phi], R[\psi_1], \dots, R[\psi_n]$. For instance, $[\Diamond b]? \rightarrow R[b], R[\Diamond b]$, because if $\Diamond b$ is undecided, we need to re-evaluate b and $\Diamond b$ at the next step.

Both active rules and truth evaluations can be enriched by a *qualifier*, such as B (binary), L (left) and R (right), which provides additional information recording which operands within a formula/subformula are still being watched. For instance, $R[a \wedge \Diamond b], [a]T \rightarrow [a \wedge \Diamond b]?R$, because if the left conjunct a is true, it is still needed to check the truth value right conjunct $\Diamond b$ to assign a truth value to the conjunction.

Verification of a trace against a formula proceeds iteratively on the cells of the trace. At the beginning of each iteration (corresponding to the monitoring of a cell), the state S contains a set of rule names corresponding to the subformulae to be checked in that cell. First, the procedure includes observations of the current cell in the state S and fires the evaluation rules repeatedly incrementing the state itself until it reaches a fixpoint. In practice, this means computing the truth values of all subformulae of ϕ in a bottom-up way, from simple atoms to ϕ itself (see Figure ...). If the root evaluates to true (resp. false), the monitor halts with verdict SATISFY (resp. VIOLATE). Otherwise, the reactivation rules are fired, and the result constitute the state of the next iteration. Note that in this case the state is not expanded, but replaced by the result of reactivation rules. This is used to flush all the previous truth evaluation, which are to be computed from scratch in the new cell. The procedure terminates with a decided verdict because of special END-triggered evaluation rules, that returns either true or false if we consumed the whole trace.

A RuleRunner system can be translated in a logic program, which in turn can be encoded in a Recurrent Neural Network with a single hidden layer, s.t. each evaluation and reactivation rule is a mapping from the input layer to the output one. For a detailed description of RuleRunner and verification algorithm see [8, 7].

Examples. A limitation of RuleRunner emerges for formulas in which a temporal operator is nested inside the operand of another temporal operator that may reactivate that operand across successive cells. RuleRunner relies on maintaining a single shared state, with one truth status for each syntactic subformula. This design is well suited to many flat temporal specifications, but in nested cases it can conflate distinct live obligations corresponding to the same subformula at different trace positions.

Consider the two formulas $\varphi_g = F(a \wedge b)$ and $\varphi_b = F(a \wedge Xb)$, which differ only by a single next operator, monitored over the same trace $\sigma = (\{a\}, \emptyset, \{b\})$. Neither formula is satisfied: φ_g would require a cell where a and b hold *together* (but a occurs only at $t = 0$ and b only at $t = 2$), and φ_b would require a cell t with $a \in \sigma_t$ and $b \in \sigma_{t+1}$ (but the only a is at $t = 0$, and is not followed by a b at $t = 1$). The correct verdict in both cases is therefore VIOLATE, but RuleRunner can provide the correct answer only for the first formula.

First, consider φ_g . The run is:

t	σ_t	evaluation	reactivation
0	$\{a\}$	$[a]T, [b]F \rightarrow [a \wedge b]F$ $[a \wedge b]F \rightarrow [F(a \wedge b)]?$	$\rightarrow R[F(a \wedge b)], R[a], R[b], R[a \wedge b]$
1	$\emptyset$	$[a]F, [b]F \rightarrow [a \wedge b]F$ $[a \wedge b]F \rightarrow [F(a \wedge b)]?$	$\rightarrow R[F(a \wedge b)], R[a], R[b], R[a \wedge b]$
2	$\{b\}$	$[a]F, [b]T \rightarrow [a \wedge b]F$ $[a \wedge b]F \rightarrow [F(a \wedge b)]?$ $[F(a \wedge b)]? \rightarrow_{\text{END}} [F(a \wedge b)]F$	$\rightarrow \text{VIOLATE}$

Now, consider φ_b . In this case, the run diverges:

t	σ_t	evaluation	reactivation
0	$\{a\}$	$R[Xb] \rightarrow [Xb]?$ $[a]T, [Xb]? \rightarrow [a \wedge Xb]?R$ $[a \wedge Xb]?R \rightarrow [F(a \wedge Xb)]?$	$\rightarrow R[Xb]M, R[b]$ $\rightarrow R[a \wedge Xb]R$ $\rightarrow R[F(a \wedge Xb)], R[a], R[Xb], R[a \wedge Xb]$
1	$\emptyset$	$R[Xb] \rightarrow [Xb]?$ $R[Xb]M, [b]F \rightarrow [Xb]F$ $R[a \wedge Xb]R, [Xb]? \rightarrow [a \wedge Xb]?R$ $R[a \wedge Xb]R, [Xb]T \rightarrow [a \wedge Xb]T$ $R[a \wedge Xb], [a]F \rightarrow [a \wedge Xb]F$ $[a \wedge Xb]?R \rightarrow [F(a \wedge Xb)]?$	$\rightarrow R[Xb]M, R[b]$ $\rightarrow R[a \wedge Xb]R$ $\rightarrow R[F(a \wedge Xb)], R[a], R[Xb], R[a \wedge Xb]$
2	$\{b\}$	$R[Xb] \rightarrow [Xb]?$ $R[Xb]M, [b]T \rightarrow [Xb]T$ $R[a \wedge Xb]R, [Xb]T \rightarrow [a \wedge Xb]T$ $R[a \wedge Xb], [a]F \rightarrow [a \wedge Xb]F$ $[a \wedge Xb]T \rightarrow [F(a \wedge Xb)]T$	$\rightarrow \text{SATISFY (wrong)}$

The divergence stems from RuleRunner's single shared state, which keeps exactly one truth slot per syntactic subformula. For φ_g the operand $a \wedge b$ is purely propositional: it is re-evaluated from scratch at every cell and never carried across cells, so the conjunction always reflects the current instant alone, and the monitor correctly reports VIOLATE. For φ_b , instead, the operand contains the temporal subformula Xb , whose truth is only settled one cell later; the eventually operator

spawns a fresh copy of $a \wedge Xb$ at every cell, yet all copies share the single literal $[Xb]$ and the single carried conjunction $[a \wedge Xb]R$. When b finally holds at $t = 2$, the resulting $[Xb]T$ is consumed by the carried mode- R conjunction, which pins its already-settled left conjunct to true and concludes $[a \wedge Xb]T$ — even though no single instant ever had a immediately followed by b . Lacking any way to distinguish obligations originating at different positions, RuleRunner conflates them and wrongly reports SATISFY. This is a structural consequence of the one-literal-per-subformula encoding, not an implementation artifact, and it affects exactly the nested-temporal formulas (such as the response pattern $G(a \rightarrow Fb)$) that are common in practice; the automata-based paradigms of the next section do not suffer from it, since a single canonical state machine never conflates concurrent obligations.

3.3 Automata-based NeSy Monitoring: DeepDFA

Architecture. DeepDFA [11] keeps the automaton of the symbolic paradigm but replaces its explicit state walk with a differentiable, tensorized one; we adopt the same forward formulation used for predictive monitoring of business processes by [6]. Starting again from the minimal DFA $A_\varphi = (2^\Sigma, Q, q_0, \delta, F)$ of Section 2.2, we encode its transition function as a three-way tensor $T \in \{0, 1\}^{|Q| \times |2^\Sigma| \times |Q|}$ with $T[q, \sigma, q'] = 1$ iff $\delta(q, \sigma) = q'$, and the accepting set as an indicator vector $v_o \in \{0, 1\}^{|Q|}$ with $v_o[q] = 1$ iff $q \in F$. The monitor state is no longer a single automaton state but a row vector $\mathbf{q}_t \in [0, 1]^{|Q|}$ over states, initialized to the one-hot vector $\mathbf{q}_0 = v_i$ of q_0 . Writing $T_\sigma = T[\cdot, \sigma, \cdot] \in \{0, 1\}^{|Q| \times |Q|}$ for the (row-stochastic) transition matrix of symbol σ , a step is a single vector-matrix product

$$\mathbf{q}_t = \mathbf{q}_{t-1} T_{\sigma_t}, \quad \text{acc}_t = \mathbf{q}_t \cdot v_o, \quad (3)$$

where $\text{acc}_t \in [0, 1]$ is the acceptance score of the prefix read so far. On crisp observations every T_{σ_t} is a 0/1 permutation-like matrix and $\mathbf{q}_t$ stays one-hot, so the update reproduces the symbolic walk exactly: $\mathbf{q}_t$ is the indicator of $\delta^*(q_0, \sigma_1 \cdots \sigma_t)$. Consequently DeepDFA is, like the symbolic monitor and unlike RuleRunner, *exact on every LTL_f formula*, including the nested-temporal specifications of Section 3.2; the single canonical automaton again precludes the conflation of concurrent obligations. The three-valued verdict and early termination are read off exactly as in Section 3.1, by testing membership of $\text{argmax}_q \mathbf{q}_t[q]$ in the precomputed trap and accepting-sink sets, and the final verdict tests its acceptance.

Soft observations and differentiability. The reason to pay for this tensorization is that Equation (3) extends verbatim to *probabilistic* observations, which the symbolic walk cannot represent. Suppose a cell is delivered not as a crisp assignment but as a vector of per-atom probabilities $\mathbf{p}_t \in [0, 1]^\Sigma$, as produced by a neural perceptor or a noisy sensor. We replace the one-hot symbol selection by an *expected* transition matrix $M(\mathbf{p}_t) \in [0, 1]^{|Q| \times |Q|}$, whose entry (q, q') is the probability that the guard of edge $q \rightarrow q'$ is satisfied under $\mathbf{p}_t$, computed compositionally over the boolean structure of the guard under an atom-independence assumption ($P(a) = p_a$, $P(\neg\psi) = 1 - P(\psi)$, $P(\psi_1 \wedge \psi_2) = P(\psi_1)P(\psi_2)$, $P(\psi_1 \vee \psi_2) = 1 - (1 - P(\psi_1))(1 - P(\psi_2))$). The update and acceptance score are unchanged,

$$\mathbf{q}_t = \mathbf{q}_{t-1} M(\mathbf{p}_t), \quad \text{acc}_t = \mathbf{q}_t \cdot v_o, \quad (4)$$

but now $\mathbf{q}_t$ is a genuine distribution over DFA states and acc_t is a calibrated acceptance probability that propagates the perceptual uncertainty all the way to the verdict, rather than discarding it through a hard 0.5 threshold. Because each $M(\mathbf{p}_t)$ is a smooth, row-stochastic function of $\mathbf{p}_t$ (and,

if desired, of learnable parameters of T), the whole monitor is differentiable: gradients of a verdict loss flow back through the automaton, which is what makes DeepDFA the vehicle for gradient-based specification adaptation and, ultimately, end-to-end training of a perceptor. On crisp inputs $M(\mathbf{p}_t)$ collapses to T_{σ_t} and the probabilistic semantics is exact for any guard; on fractional inputs it is exact whenever the guard is *read-once* (each atom occurring once), and an independence approximation otherwise.

The alphabet blow-up. DeepDFA’s structural cost is dual to those of the other two paradigms. The transition tensor is indexed by the full alphabet 2^Σ , so a dense materialization is exponential in the number of atoms, $|T| = |Q|^2 \cdot 2^{|\Sigma|}$. One can avoid building the dense tensor by evaluating the per-edge guard probabilities on the fly — the factored form underlying Equations (3)–(4), which never enumerates 2^Σ — and this is what we use for specifications over many atoms. But the blow-up is not merely an implementation choice: when a single guard depends on all $|\Sigma|$ atoms there is in general no sub-exponential set of symbols to sum over, and only the read-once circuit structure of a guard (which a flat DFA does not expose) permits its satisfaction probability to be computed in time linear in $|\Sigma|$. This alphabet blow-up is therefore DeepDFA’s Achilles heel, the counterpart of the symbolic monitor’s state-space blow-up and RuleRunner’s nested-temporal limit, and we return to it in the comparison of Section 4 and the scalability experiments of Section 5.

4 Theoretical Comparison

5 Experimental Comparison

All three monitoring paradigms are implemented in a common framework sharing a uniform MONITOR interface, which ensures that the experimental harness is paradigm-agnostic: timing experiments iterate over a list of monitor instances without any paradigm-specific branching. The shared compilation pipeline uses `ltl2dfa` [3] to translate each LTL_f formula into a minimal DFA once at construction time.

We evaluate the monitoring paradigms along three axes, using synthetic traces throughout Experiments 1–3 to isolate per-step monitoring cost from data-dependent effects such as early-termination frequency. This follows the methodology of experiments in [8].

6 Related Work

7 Conclusion and Future Work

In this paper ...

7.1 Transformers

7.2 Specification Adaptation

7.3 Process Model Repair

References

- [1] Andreas Bauer, Martin Leucker, and Christian Schallhart. Runtime verification for LTL and TLTL. *ACM Trans. Softw. Eng. Methodol.*, 20(4):14:1–14:64, 2011.
- [2] Giuseppe De Giacomo and Moshe Y. Vardi. Linear temporal logic and linear dynamic logic on finite traces. In Francesca Rossi, editor, *IJCAI 2013, Proceedings of the 23rd International Joint Conference on Artificial Intelligence, Beijing, China, August 3-9, 2013*, pages 854–860. IJCAI/AAAI, 2013.
- [3] Francesco Fuggitti. Ltlf2dfa. *Zenodo*, 2019.
- [4] Orna Lichtenstein, Amir Pnueli, and Lenore Zuck. The glory of the past. In *Workshop on Logic of Programs*, pages 196–218. Springer, 1985.
- [5] Zohar Manna and Amir Pnueli. *Temporal Verification of Reactive Systems*, volume 1. Springer-Verlag, 1995.
- [6] Axel Mezini, Elena Umili, Ivan Donadello, Fabrizio Maria Maggi, Matteo Mancanelli, and Fabio Patrizi. Neuro-symbolic predictive process monitoring. *arXiv preprint arXiv:2509.00834*, 2025.
- [7] Alan Perotti, Guido Boella, and Artur d’Avila Garcez. Runtime verification through forward chaining. In *Runtime Verification: 6th International Conference, RV 2015, Vienna, Austria, September 22-25, 2015. Proceedings*, pages 185–200. Springer, 2015.
- [8] Alan Perotti, Artur d’Avila Garcez, and Guido Boella. Neural networks for runtime verification. In *2014 International Joint Conference on Neural Networks (IJCNN)*, pages 2637–2644. IEEE, 2014.
- [9] Alan Perotti, Artur d’Avila Garcez, and Guido Boella. Neural-symbolic monitoring and adaptation. In *2015 International Joint Conference on Neural Networks (IJCNN)*, pages 1–8. IEEE, 2015.
- [10] Amir Pnueli. The temporal logic of programs. In *18th Annual Symposium on Foundations of Computer Science, Providence, Rhode Island, USA, 31 October - 1 November 1977*, pages 46–57. IEEE Computer Society, 1977.
- [11] Elena Umili and Roberto Capobianco. Deepdfa: Automata learning through neural probabilistic relaxations. In *ECAI 2024 - 27th European Conference on Artificial Intelligence, 19-24 October 2024, Santiago de Compostela, Spain - Including 13th Conference on Prestigious Applications of Intelligent Systems (PAIS 2024)*, pages 1051–1058, 2024.
- [12] Shufang Zhu, Lucas M. Tabajara, Jianwen Li, Geguang Pu, and Moshe Y. Vardi. Symbolic LTLf synthesis. In Carles Sierra, editor, *Proceedings of the Twenty-Sixth International Joint Conference on Artificial Intelligence, IJCAI 2017, Melbourne, Australia, August 19-25, 2017*, pages 1362–1369. ijcai.org, 2017.